

Paidiverpy: A Python package designed to create pipelines for preprocessing image data for biodiversity analysis

Tobias Ferreira¹, Mojtaba Masoudi¹, Audenhaege van Loic¹, Erik Orenstein¹, Colin Sauze¹, and Jennifer Durden¹

¹ National Oceanography Centre, UK

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Paidiverpy is an open-source Python package for constructing reproducible image-preprocessing workflows for biodiversity research. It provides a configurable, layer-based pipeline for combining image conversion, enhancement, spatial processing, sampling, metadata management, and custom processing operations while preserving the relationship between imagery and associated contextual information. Workflows can be defined declaratively, executed locally or using parallel computing resources, inspected at intermediate stages, and reused across datasets. By integrating established image-processing libraries, biodiversity-oriented metadata standards, scalable execution, and multiple user interfaces within a common workflow framework, Paidiverpy aims to reduce the technical effort required to transform heterogeneous scientific imagery into documented, reproducible, analysis-ready datasets for ecological analysis and automated image annotation.

Statement of Need

Imaging technologies are increasingly used for biodiversity observation. Advances in digital cameras, autonomous platforms, microscopy, and related systems enable image collection at scales that were previously impractical (Durden & others, 2016). However, reliable ecological analysis still depends on consistent and scientifically appropriate preprocessing (Young et al., 2017).

Preprocessing is important because acquisition variability can affect the detectability and comparability of biological features. Common operations include format conversion, enhancement, geometric correction, image selection, sampling control, and metadata standardisation (Durden & others, 2016; Song et al., 2022; Young et al., 2017). Several characteristics make this challenging:

- Heterogeneous image sources:** Datasets for biodiversity analysis are collected using diverse instruments, platforms, and environmental conditions, resulting in variation in formats, resolution, and preprocessing requirements (Durden & others, 2016; Pettorelli et al., 2014).
- Inconsistent metadata handling:** Metadata describing when, where, and how images were acquired may be distributed across files, tables, and sensor records. Standards such as image FAIR Digital Objects (iFDOs) improve interoperability and reuse (Schoening et al., 2022; Wilkinson & others, 2016).
- Complex preprocessing requirements:** Biodiversity imagery often requires multiple interacting operations whose selection and order depend on the dataset and research

objectives (Song et al., 2022; Young et al., 2017). Poor choices can affect downstream interpretation (Young et al., 2017).

- Scalability and computational constraints: Large, high-resolution collections can make preprocessing computationally demanding, particularly when methods and parameters require repeated testing (Durden & others, 2016; Song et al., 2024).
- Repeatability and provenance: Recording processing steps, parameters, and metadata transformations is important for reproducibility and reuse across datasets and research groups (Schoening et al., 2022; Wilkinson & others, 2016).

Paidiverpy addresses these requirements by explicitly representing preprocessing operations, metadata, and workflow configuration. Rather than replacing existing image-processing libraries, it provides an orchestration layer for applying them consistently within reproducible biodiversity-image workflows.

State of the Field

Mature Python libraries already provide many relevant image-processing algorithms, like OpenCV (OpenCV Library, n.d.), scikit-image (Walt & others, 2014), and Pillow (Pillow Library, n.d.). Paidiverpy builds on these libraries rather than reimplementing their functionality. However, they mainly provide operations on individual images or arrays, leaving researchers to manage workflow order, configuration, metadata, and consistent processing across collections.

Scientific workflow systems can provide more general mechanisms for defining reproducible computational pipelines, while distributed-computing frameworks such as Dask provide scalable execution. These systems address broader workflow and execution problems but do not themselves provide a biodiversity-image model combining image preprocessing, scientific metadata, image sampling, and biodiversity-specific workflow requirements.

Paidiverpy fills this gap by combining established image-processing tools within a domain-oriented framework for defining, validating, and executing reproducible preprocessing workflows.

Software Design

Paidiverpy was designed around the separation of workflow orchestration from image-processing algorithms. Rather than implementing a new image-processing library, it integrates established tools and exposes their functionality through a common pipeline abstraction. This choice reduces duplication of mature software while allowing preprocessing operations to be represented consistently, combined into reusable workflows, and extended with domain or project-specific algorithms.

The central abstraction is a sequence of processing layers. Operations are grouped into components including OpenLayer, ConvertLayer, ColourLayer, PositionLayer, SamplingLayer, and CustomLayer. A central Pipeline coordinates these layers and maintains the dataset as it moves through the workflow. The layer architecture provides a common lifecycle for otherwise heterogeneous operations and makes it possible to inspect, replace, reorder or extend individual stages without rewriting the complete pipeline.

A second design decision is to make the workflow itself a portable research object. Pipelines can be described using YAML configuration files rather than solely through Python code. Parameters and processing order can therefore be reviewed independently of the implementation, version controlled alongside analyses, and reapplied to another dataset. The configuration is validated before execution, reducing the likelihood that differences between runs arise from silently missing or inconsistent parameters.

Metadata are treated as part of the scientific dataset rather than auxiliary files. Paidiverpy supports tabular, and iFDO-compatible metadata (Schoening et al., 2022), allowing contextual

86 information to remain associated with images while preprocessing steps are executed. This
87 is particularly important for biodiversity imagery because variables such as acquisition time,
88 position, camera orientation and sampling conditions may determine which images should be
89 retained or how they should be processed.

90 Scalability is provided through optional integration with Dask, rather than by implementing a
91 bespoke parallel-computing system. The same high-level pipeline abstraction can therefore be
92 used for local execution and parallel processing. This design keeps domain-specific workflow
93 logic within Paidiverpy while delegating scheduling and parallel execution to an established
94 ecosystem.

95 Extensibility was another explicit requirement. The CustomLayer mechanism allows researchers
96 to introduce project-specific preprocessing algorithms while preserving the surrounding workflow,
97 configuration and metadata-management infrastructure.

98 Paidiverpy can be used as a Python library, through a command-line interface or web
99 application, and in containerised environments, all using the same underlying processing model.
100 The project is openly developed on GitHub under the Apache-2.0 licence, with documentation
101 on ReadTheDocs, pip installation, Docker images, and example datasets and notebooks
102 demonstrating key workflows such as pipeline creation, parallel execution, object-store access,
103 custom algorithms, RAW image processing, and metadata handling.

104 Research Impact Statement

105 Paidiverpy is used within marine-imaging research workflows at the National Oceanography
106 Centre and has begun to be adopted beyond its original development team, supporting the
107 preparation and standardisation of imagery for biodiversity analysis and automated classification.

108 One external application is the DEAL (DEcentrAlised Learning for automated image analysis
109 and biodiversity monitoring) project, led by Plymouth Marine Laboratory. DEAL develops
110 collaborative machine-learning approaches for plankton and seafloor imagery and uses
111 Paidiverpy for metadata handling and standardisation within its workflows.

112 Paidiverpy is also being evaluated/used alongside deep-frameX, an open-source tool for
113 extracting scientific frames from deep-sea video. deep-frameX produces self-describing frames
114 with embedded metadata and iFDO manifests, providing a complementary workflow in
115 which frame extraction and metadata management precede preprocessing and analysis with
116 Paidiverpy.

117 Within the National Oceanography Centre, Paidiverpy is used for XXXXXXXXXXXXXXXX,
118 providing a common workflow for preprocessing [benthic/plankton/pelagic/etc.] imagery.
119 These applications have also informed example datasets and best-practice guidance for
120 biodiversity image preprocessing.

121 The package is intended as reusable infrastructure between image acquisition and downstream
122 manual or automated interpretation, rather than software tied to a single dataset or analysis.

123 Conclusion

124 Paidiverpy provides a flexible and extensible framework for preprocessing ecological image
125 datasets. Its layer-based design, explicit workflow definitions, metadata integration, and
126 support for parallel execution address key limitations in existing image-based biodiversity
127 analysis workflows. The package enables users to construct transparent preprocessing pipelines
128 and generate analysis-ready datasets for downstream ecological and AI-based applications.

129 The software is openly available and distributed with example datasets, pre-built Docker images,
130 and validation tools to support adoption. By standardizing image preprocessing workflows and

improving repeatability, Paidiverpy contributes a reusable software component for large-scale biodiversity monitoring and ecological research.

Acknowledgements

This project was supported by the UK Natural Environment Research Council (NERC) through the *Tools for automating image analysis for biodiversity monitoring (AIAB)* Funding Opportunity, reference code **UKRI052**.

References

- Durden, J. M., & others. (2016). Perspectives in visual imaging for marine biology and ecology. *Oceanography and Marine Biology: An Annual Review*, 54. <https://doi.org/10.1201/9781315368597>
- OpenCV library*. (n.d.). <https://opencv.org/>.
- Pettorelli, N., Laurance, W. F., O'Brien, T. G., Wegmann, M., Nagendra, H., Turner, W., & Milner-Gulland, E. J. (2014). Satellite remote sensing for applied ecologists: Opportunities and challenges. *Journal of Applied Ecology*, 51(4), 839–848. <https://doi.org/10.1111/1365-2664.12261>
- Pillow library*. (n.d.). <https://pillow.readthedocs.io/>.
- Schoening, T., Durden, J. M., Faber, C., Felden, J., Heger, K., Hoving, H.-J. T., Kiko, R., Köser, K., Krammer, C., Kwasnitschka, T., Möller, K. O., Nakath, D., Nass, A., Nattkemper, T. W., Purser, A., & Zurowietz, M. (2022). Making marine image data FAIR. *Scientific Data*, 9, 414. <https://doi.org/10.1038/s41597-022-01491-3>
- Song, Y., Nakath, D., She, M., & Köser, K. (2022). Optical imaging and image restoration techniques for deep ocean mapping: A comprehensive survey. *PFG – Journal of Photogrammetry, Remote Sensing and Geoinformation Science*, 90(3), 243–267. <https://doi.org/10.1007/s41064-022-00206-y>
- Song, Y., She, M., & Köser, K. (2024). Advanced underwater image restoration in complex illumination conditions. *ISPRS Journal of Photogrammetry and Remote Sensing*, 209, 197–212. <https://doi.org/10.1016/j.isprsjprs.2024.02.004>
- Walt, S. van der, & others. (2014). Scikit-image: Image processing in python. *PeerJ*, 2, e453. <https://doi.org/10.7717/peerj.453>
- Wilkinson, M. D., & others. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Young, N. E., Anderson, R. S., Chignell, S. M., Vorster, A. G., Lawrence, R., & Evangelista, P. H. (2017). A survival guide to landsat preprocessing. *Ecology*, 98(4), 920–932. <https://doi.org/https://doi.org/10.1002/ecy.1730>